process. This requires you to pass the PID of the current process as part of the message and then to receive a response. That's pretty cumbersome, but by the end of this chapter, you'll see how GenServer makes this easy.

Our process is no longer alive after a single message—it terminated after its code finished executing. Let's make it process messages forever.

Process Messages Forever

Recursion is very useful to create infinite loops. Usually, an infinite loop would be a bad thing, but it's totally fine when used in a controlled way.

Create lib/examples/spawn/infinite.ex and add the following code:

elixir_examples/lib/examples/spawn/infinite.ex

```
defmodule Examples.Spawn.Infinite do
  def start do
    spawn(& loop/0)
  end

  defp loop do
    receive do
      {:add, a, b} ->
        IO.puts(a + b)
➤       loop()

      :memory ->
        {:memory, bytes} = Process.info(self(), :memory)
        IO.puts("I am using #{bytes} bytes")
➤       loop()

      :crash ->
        raise "I crashed"

      :bye ->
        IO.puts("Goodbye")
    end
  end
end
```

The start/0 function uses spawn/1 to kick off our looped process. The loop function is simple—it's just a receive block with a variety of messages handled. For all messages, except :bye, the loop/0 function is called as the last thing the function does. This creates a recursive loop that will handle messages forever. Let's try it out:

```
$ iex -S mix
iex> pid = Examples.Spawn.Infinite.start()

iex> send(pid, :memory)
I am using 2608 bytes
```